ACTIONABLE LINKS

Transforming Documentation into Live Web Walkthroughs

AI-Driven Interactive Action Guidance Engine & Chrome Overlay Player

STARTUP BUSINESS & TECHNICAL REPORT

Actionable Links solves the user onboarding and software guidance crisis. By dynamically parsing web layouts and visual documentation via Gemini 2.5 Flash, the platform converts YouTube videos, static guides, and recording files into interactive, real-time DOM walkthroughs. Built with a dual-engine architecture consisting of a Next.js Creator Console and a lightweight Shadow-DOM isolated Chrome Extension, it offers instant guidance overlays directly on target web interfaces without a single line of customer integration code.

Key Platform Advantages:

- Zero-code Client Integration: Renders overlays on any SaaS web portal via Chrome Extension.
- AI-Generated Guides: Converts YouTube and documentation into step actions in under 10 seconds.
- Precision Spotlight Overlay: Isolates elements using custom SVG masking and real-time DOM polling.
- Enterprise-ready Backend: Integrated with Supabase auth, progress synchronization, and RLS rules.

PROJECT STATUS:	Prototype Complete
TECH STACK:	Next.js 15, Vite, React 18, Supabase SSR, Gemini 2.5 Flash, TypeScript
DEPLOYMENT SLATE:	Vercel + Chrome Web Store + Supabase Cloud
AUTHOR/TEAM:	Team Local Host
DATE:	July 2, 2026

1. Comprehensive Technology Stack

Component	Technology	Role & Engineering Decisions
Creator App	Next.js 15 (App Router)	Enables server components, fast routing, and Server Actions for tutorial CRUD. Lowers st
Extension	Vite + React + TS	Builds highly optimized, lightweight bundles. ES Modules dynamic loading handles injectio
Database	Supabase PostgreSQL	Provides scalable schema with full Row Level Security (RLS) policies. Powers rapid auth r
Authentication	Supabase SSR Auth	Unifies cookie sessions between Next.js and Chrome Extension on cross-origin API calls.
AI Orchestrator	Gemini 2.5 Flash	Extracts structured instructions and matches them to page selectors via JSON schema va
Styling	Tailwind CSS + Custom CSS	Used for both web dashboards and inside Shadow-DOM containers to isolate extension st

2. System Architecture & Flows

The system relies on asynchronous message passing and secure database gateways. The Chrome Extension functions on client pages by querying the Next.js API for the current hostname. Here is the system communications flow mapping:

```
+------------------------------------------------------------+
|                   CHROME EXTENSION PLAYER                  |
|                                                            |
|  [Popup Dashboard]                                         |
|        | (tabs.sendMessage)                                |
|        v                                                   |
|  [Content Script (Shadow DOM)] <-> [DOM Observer]          |
|        |                            (MutationObserver)     |
|        | (runtime.sendMessage)                             |
|        v                                                   |
|  [Background Service Worker] <----> [Chrome Local Storage] |
+--------+---------------------------------------------------+
         |
         | HTTP Request (with CORS & Credentials)
         v
+--------+---------------------------------------------------+
|                 NEXT.JS APP BACKEND / GATEWAY              |
|                                                            |
|  [CORS Handler] --> [Server Routes (/api/v1/*)]            |
|                            |                               |
|                            +---> [Supabase Client]         |
|                            |           | (PostgreSQL RLS)  |
|                            |           v                   |
|                            |     [(DB) Supabase Database]  |
|                            v                               |
|                      [Gemini 2.5 AI Pipeline]              |
+----------------------------+-------------------------------+
                             |
                   [Google Generative Language]
```

3. Core Engineering Highlights

» Shadow DOM Isolation

Web walkthrough engines suffer from style contamination where host website CSS breaks the overlay layouts. We solved this by attaching a Shadow Root in content.ts. All styling is loaded dynamically inside this isolated root via Web Accessible Resources, shielding the HUD player from host overrides and protecting host websites from extension side-effects.

» Pulsing Spotlight SVG Cutout

To create high-premium visual isolation without blocking interactions, we implemented an SVG-based Spotlight overlay. Rather than layering simple border highlights, the spotlight component dynamically fetches the bounding rect coordinates of the target element, pads the coordinates by 6px, and recalculates an SVG mask cutout in real-time, matching the window scroll and viewport resize events.

» Asynchronous DOM Observer

Single Page Apps (SPAs) re-render parts of the page dynamically, causing target selectors to disappear or load lazily. Our DomObserver script starts a MutationObserver listening to body child additions. If a step target selector was missing but gets injected into the DOM, the player automatically recovers and resumes highlighting, creating a seamless user experience.

» Secure Cross-Origin (CORS) Cookie Authentication

Extensions communicate with servers across domains, which browsers block by default. We custom-coded a robust CORS response mapping system inside Next.js APIs to echo the extension's origin, set Access-Control-Allow-Credentials, and read encrypted Supabase JWT cookies. This permits direct progress-syncs and active session checks directly from background service workers.

4. Real-world Deployment Roadmap

Phase 1: Foundation & Community Launch (Months 1 - 3)

Goal: Validate developer traction and product-market fit.

- Publish the extension to the Chrome Web Store with open developer access.
- Enable self-hosted Next.js application deployments via single-click Vercel integration.
- Integrate Google Gemini 2.5 Flash pipeline with rate-limiting and user API key support.
- Set up database replication and monitoring on Supabase Cloud.

Phase 2: Enterprise Security & Analytics (Months 4 - 6)

Goal: Adapt platform for corporate SaaS and team-based walkthrough management.

- Introduce SOC-2 and HIPAA compliance guidelines for DOM scraping and recording.
- Add Telemetry and Analytics dashboards: track walkthrough completion rates and drop-off steps.
- Enable Role-Based Access Control (RBAC): team workspaces, editors, and reviewers.
- Provide customized branding options: allow companies to customize spotlight colors, HUD logo, and layout fonts.

Phase 3: Advanced Automation & Marketplace (Months 7 - 12)

Goal: Transition Actionable Links into a web automation ecosystem.

- Launch a Public Guide Marketplace: allow creators to monetize tutorials for complex platforms.
- AI Auto-Healer: use Gemini to automatically fix broken CSS selectors when host websites deploy updates.
- Cross-Browser Support: release Safari (WebExtension), Firefox, and Edge equivalents.
- Action Hooks: trigger webhooks or custom code execution when specific walkthrough steps are completed.

5. Startup Monetization Strategy

Actionable Links will operate on a Freemium SaaS model. The base platform is free for open-source and individual developers (up to 3 public guides). The Enterprise Tier starts at $49/month, unlocking private guides, progress analytics, team workspaces, and AI selector-healing capabilities. Special corporate custom licensing will support on-premise configurations and private Supabase database integrations.